

BUT Science des Données - Semestre 3

COMPETENCE 4 : Développer un outil décisionnel - parcours VCOD

Collecte de données web

F.GARNIER



Plan de la Saé

Chapitre 1 : Web Scraping

Chapitre 2 : API

Sujet noté

Chapitre 2 : API

1. Introduction - API : Application Programming Interface

L'extraction d'informations à partir de pages HTML (web scraping) est une tâche compliquée et peu fiable (cf chapitre 1). Si la mise en page du site vient à changer, il est probable qu'un programme comme celui que vous venez d'écrire ne fonctionne plus.

Pour faciliter l'utilisation de leurs informations par des programmes, de nombreux sites fournissent, en plus des pages HTML, des données structurées plus adaptées pour un traitement automatique et **légal** ! On parle alors d'**API Web**.

Bon nombre de services Web proposent des API. Ces API permettront à un programme Python de récupérer et traiter un jeu de données fourni à travers une url.

En règle générale, les APIs sont documentées :

- https://developers.google.com/youtube/2.0/developers_guide_json
- <http://developer.vimeo.com/apis/>
- <https://developer.twitter.com/en/docs/twitter-api>
- <http://www.reddit.com/dev/api>
- http://wiki.openstreetmap.org/wiki/API_v0.6
- <https://code.google.com/p/google-api-python-client/>

Par exemple, avec l'API web Twitter, on peut écrire un programme Python qui collecte des données sur les tweets

2. Exemples d'utilisation

Voici quelques exemples de ce qu'il est possible de faire avec Python, en utilisant la librairie `requests` pour simplifier les requêtes Web.

2.1 OpenWeatherMap

Ce site propose une API permettant de connaître les conditions météo sur n'importe quel point du globe. La requête est de la forme :

```
http://api.openweathermap.org/data/2.5/weather?APPID=cleAPI&q=LIEU
```

Il faut au préalable créer un compte sur le site <https://home.openweathermap.org> pour obtenir une clé « API » à renseigner dans la précédente url.

Le lieu est un nom de ville suivie d'une virgule et de l'extension du pays, par exemple *Niort,fr*. On peut aussi rechercher par coordonnées (latitude, longitude) ou l'identifiant de ville. La documentation est ici <http://api.openweathermap.org/API/>

Le résultat dans ce script est obtenu au format JSON (d'autres formats sont disponibles). Tester le code suivant :

```
import requests
url="http://api.openweathermap.org/data/2.5/weather?APPID=369acb283342c0010fdeef5621dd1fc6&q=Niort,fr"
contenu = requests.get(url)
print(contenu)
data = contenu.json()
print(data)
t = data['main']['temp']
print("La température est de ",round(t-273.15,2)," degrés C")
```

Résultat :

```
{
'coord': {'lon': -0.3333, 'lat': 46.3333},
'weather': [
  {'id': 803,
   'main': 'Clouds',
   'description': 'broken clouds',
   'icon': '04d'}],
'base': 'stations',
'main': {'temp': 297.38,           # en kelvin !
         'feels_like': 297.29,
         'temp_min': 297.38,
         'temp_max': 297.43,
         'pressure': 1011,
         'humidity': 55},
'visibility': 10000,
'wind': {'speed': 1.79, 'deg': 262, 'gust': 5.36},
'clouds': {'all': 84},
'dt': 1660747795,
'sys': {'type': 2,
        'id': 2039202,
        'country': 'FR',
        'sunrise': 1660712549,
        'sunset': 1660763321},
'timezone': 7200,
'id': 2990354,
'name': 'Arrondissement de Niort',
'cod': 200
}
```

La température est de 24.23 degrés C # cours préparé le 17 août à 17h08 😊

2.2 OpenFoodFacts

Le site <http://fr.openfoodfacts.org/> recense des données sur les produits alimentaires. L'alimentation de la base est faite grâce au crowdsourcing et les données sont librement accessibles.

On peut interroger la base à partir du code barre d'un produit :

<http://fr.openfoodfacts.org/api/v0/produit/3029330003533.json>

Plus d'infos ici : <http://fr.openfoodfacts.org/data>

⇒ **En réutilisant le script écrit pour l'API d'openWeatherMap, écrire un script python qui affiche l'abréviation du nom en français du produit dont le lien a été donné ci-dessus.**

2.3 API Velib Paris -> Aller plus loin sur le filtrage des données via l'URL de l'API

Open Data Soft a développé un outil de gestion des données ouvertes, qui est de plus en plus utilisé, en particulier par la mairie de Paris. Vous pouvez trouver l'aide en ligne à cette adresse :

https://help.opendatasoft.com/apis/ods-explore-v2/explore_v2.1.html

Nous allons interroger l'API Velib, proposée par la mairie de Paris. Toutes les informations sont disponibles à cette adresse : <https://opendata.paris.fr/explore/dataset/velib-disponibilite-en-temps-reel/api/>

Premier essai : L'interrogation se fait, sous Python, avec la fonction `get()` de la librairie `requests`. Le résultat peut être transformé en JSON (en dictionnaire sous Python) grâce à la fonction `json()` sur celui-ci.

On obtient un objet ayant deux informations (si tout va bien) :

- `total_count` : le nombre d'éléments correspondant à la requête (le nombre de stations) ;
- `results` : un tableau contenant une partie des résultats.

```
import requests
url_base = "https://opendata.paris.fr/api/explore/v2.1/catalog/datasets/velib-disponibilite-en-temps-reel/records"
resultat = requests.get(url_base).json()
print(resultat)
```

```
⇒ {'total_count': 1486,
    'results': [{'stationcode': '16107',
                  'name': 'Benjamin Godard - Victor Hugo',
                  'is_installed': 'OUI',
                  'capacity': 35,
                  'numdocksavailable': 28,
                  ...}]
```

Par défaut, la limite est de 10 éléments (visualiser le contenu de la variable `resultat`)

Et si on essaie de tout récupérer

La première idée est de mettre le paramètre `limit` au maximum de ce qu'on doit obtenir comme résultat, pour tout récupérer d'un coup. Comme vous pouvez le voir ci-dessous, il n'est pas possible de demander plus de 100 éléments en une fois (sauf certaines opérations que l'on verra plus loin) :

```
requests.get(url_base + "?limit=" + str(resultat["total_count"])).json()
⇒ {'error_code': 'InvalidRESTParameterError',
    'message': 'Invalid value for limit API parameter: 1486 was found but -1 <= limit <= 100 is expected.'}
```

Comment tout obtenir alors ?

Pour récupérer tous les éléments dans un tel cas, il est nécessaire de faire une boucle et d'utiliser le paramètre `offset` qui permet de passer les premiers résultats.

Le code ci-dessous permet donc de récupérer toutes les stations :

```
resultat_final = []
for i in range(0, resultat["total_count"], 100):
    temp = requests.get(url_base + "?limit=100&offset=" + str(i)).json()
    resultat_final += temp["results"]
```

```
# on trouve bien nos 1484 résultats
print(len(resultat_final))
⇒ 1486
```

Que faire des résultats ?

Pour nos besoins ultérieurs (en analyse, calculs, graphiques...), il est intéressant de voir que l'on peut transformer ce résultat JSON en un dataframe pandas très facilement :

```
import pandas
data = pandas.DataFrame(resultat_final)
```

⇒ Vérifier dans l'explorateur de variables Spyder le nombre de lignes et de colonnes.
 ⇒ **1486 rows × 14 columns**

#Analyse du dataframe :

```
print(data.groupby("stationcode").size().sort_values(ascending=False))
```

En analysant ce résultat, on peut déjà voir que l'on a des stations en double, voire en triple...

```
⇒ stationcode
   12018      3
   15071      2
   5034       2
   3007       2
   3006       2
      ..
   17105      1
   17104      1
   17102      1
```

Voici les différentes colonnes pour chaque station :

```
print(data.columns)
⇒ Index(['stationcode',      'name',      'is_installed',      'capacity',
        'numdocksavailable',
        'numbikesavailable', 'mechanical', 'ebike', 'is_renting',
        'is_returning', 'duedate', 'coordonnees_geo',
        'nom_arrondissement_communes', 'code_insee_commune'],
        dtype='object')
```

Filtrer le jeu de données via l'URL de l'API

Premières étapes classiques de l'interrogation d'une base de données, quel que soit son format (relationnel, NoSQL, autre), la restriction (sélection d'éléments sur la base d'une condition) et la projection (sélection de certaines variables) sont bien évidemment possibles avec cet outil.

Projection : Ici, on ne sélectionne que les codes des stations (stationcode) et le nom de celles-ci (name): `print(requests.get(url_base + "?select=stationcode,name").json())`

Sélection (Restriction)

On peut donc aussi faire une restriction avec la clause where (on voit qu'on est très proche du langage SQL). On cherche ici les stations avec une capacité d'accueil supérieure à 30 :

```
requests.get(url_base + "?select=stationcode,capacity&where=capacity>30").json()
```

Avec tri du résultat

Toujours dans la même idée de faire les mêmes opérations qu'en SQL, on peut trier le résultat avec la clause order_by. Même résultat que précédemment, mais trié par ordre croissant de la capacité :

```
requests.get(url_base+"?select=stationcode,capacity&where=capacity>30&order_by=capacity")
.json()
```

Et ce tri peut bien évidemment être décroissant en ajoutant DESC après le critère de tri.

Toujours même résultat, mais trié par ordre décroissant de la capacité :

```
requests.get(url_base +
"?select=stationcode,capacity&where=capacity>30&order_by=capacity DESC").json()
```

Combinaisons de conditions

On peut bien évidemment combiner plusieurs conditions avec les différents opérateurs logiques : AND, OR et NOT. Nous avons ici les stations avec une capacité supérieure à 20, qui ne sont pas en état de laisser la possibilité de louer les vélos :

```
requests.get(url_base + "?select=stationcode,capacity,is_renting&where=capacity>20
AND is_renting='NON'&limit=100").json()
```

Recherche dans une chaîne de caractères

On a trois fonctions qui permettent de chercher un chaîne (ou sous-chaîne) de caractères dans une variable (ou même globalement en indiquant *), ces premières étant sensibles à la casse :

- `startswith(champs, 'chaîne')` : cherche les éléments pour lesquels le champs cité débute par la chaîne
- `suggest(champs, 'chaîne')` : cherche les éléments pour lesquels le champs cité contient la chaîne
- `search(champs, 'chaîne')` : cherche les éléments pour lesquels le champs cité est exactement égal à la chaîne

On veut toutes les stations dont le nom commence par “Exelmans”.

```
requests.get(url_base + "?select=stationcode,name&where=startswith(name,'Exelmans')").json()
```

Recherche dans une liste

On dispose de l’opérateur IN (val1, val2, ...) permettant de tester si un champs a une valeur comprise dans la liste passée à la suite. On souhaite obtenir les stations dans les villes de Clichy et Colombes :

```
requests.get(url_base +
"?select=stationcode,nom_arrondissement_communes&where=nom_arrondissement_c
ommunes IN ('Clichy','Colombes')").json()
```

Distance à un objet géométrique

Un des points cruciaux actuellement est la géo-localisation, en particulier dans le cadre d’une application mobile. On peut ainsi requêter la base de données en cherchant, dans ce premier exemple, les éléments pour lesquels la distance entre un point géographique (stocké dans un champs) et un objet géométrique est inférieure à une certaine distance passée en paramètre. Ici, nous cherchons les stations à moins de 800m de l’IUT Paris-Rives de Seine. Il faut noter que l’ordre des coordonnées (ici longitude et latitude) dépend de la façon dont elles sont stockées dans le champs :

```
requests.get(url_base +
"?select=stationcode,name&where=within_distance(coordonnees_geo,
geom'POINT(2.267888940737877 48.84197963193564)', 800m)").json()
```

En complément, on peut en plus récupérer la distance calculée entre ces coordonnées et un objet géométrique :

```
requests.get(url_base + "?select=stationcode,name,distance(coordonnees_geo,
geom'POINT(2.267888940737877 48.84197963193564)') as
distance&where=within_distance(coordonnees_geo,
geom'POINT(2.267888940737877 48.84197963193564)', 800m)").json()
```

Comparaison à une zone géographique

La fonction `in_bbox()` teste si un champs est inclus dans une zone géographique rectangulaire, délimitée par deux points. Ici, on récupère les stations dans un rectangle contenant l'IUT.

```
requests.get(url_base +
"?select=stationcode,name&where=in_bbox(coordonnees_geo,48.84,2.26,48.85,2.27)") .json()
```

Et ici, celles qui n'y sont pas donc.

```
requests.get(url_base +
"?select=stationcode,name&where=not(in_bbox(coordonnees_geo,48.84,2.26,48.85,2.27))") .json()
```

A noter qu'il existe une fonction permettant de chercher si un point appartient à une zone géographique de type polygone.

Calcul d'agrégat

On peut aussi demander à l'API de faire un certain nombre de calculs en amont, en particulier des calculs d'agrégats, avec en plus la clause `group_by` qui permet de faire ce calcul pour chaque modalité d'un champs. Ici, on a le nombre de stations par ville et la capacité totale de celles-ci (10 villes seulement affichées pour raison de place) :

```
requests.get(url_base +
"?select=nom_arrondissement_communes,count(*),sum(capacity)&group_by=nom_arrondissement_communes&limit=10") .json()
```

On peut bien évidemment ordonner ce résultat pour avoir les villes ayant le plus de stations en premier :

```
requests.get(url_base + "?select=nom_arrondissement_communes,count(*) as nb_stations,sum(capacity)&group_by=nom_arrondissement_communes&order_by=nb_stations DESC&limit=10") .json()
```

Fonctions spéciales de découpage

Découpage d'un champs en intervalles : La fonction `RANGE()` permet de transformer un champs contenant une valeur quantitative en plusieurs modalités, en créant des intervalles sur ces valeurs.

Elle a deux fonctionnements possibles :

- par intervalles de même taille, en passant en paramètre un seul entier, qui sera la taille des intervalles créés
- par intervalles définies explicitement, en passant en paramètre la liste des bornes des intervalles

On répartit ici les stations sur la base de leur capacité, en créant des intervalles de taille 15 :

```
requests.get(url_base +
"?select=count(*)&group_by=RANGE(capacity,15)") .json()
```

Ici, on crée nous-même les intervalles (du minimum – avec * – à 14, de 15 à 19, de 20 à 25, de 25 à 30, de 30 à 40, et de 40 au maximum – avec * encore). On renomme aussi le résultat avec la clause `as` :

```
requests.get(url_base +
"?select=count(*)&group_by=RANGE(capacity,*,15,20,25,30,40,*) as capacity") .json()
```

Il faut noter que ce découpage peut aussi se faire sur une date, en se basant sur une unité pour le découpage (chaque jour, chaque année, chaque heure...).

Découpage selon un niveau de zoom : La fonction GEO_CLUSTER() permet de répartir les éléments, géo-localisés par un champs spécifié, en fonction d'un niveau de zoom défini (entre 1 et 25). Nous découpons ici les stations sur la base d'un niveau de zoom égal à 10, ce qui crée 14 clusters de stations. On récupère de plus les centres de ces clusters.

```
requests.get(url_base +
"?select=count(*)&group_by=GEO_CLUSTER(coordonnees_geo,10)").json()
```

3. Test d'API

Il est plutôt aisé de tester des API avec des outils comme Postman <https://www.postman.com/>. Cet outil permet de tester des requêtes HTTP avec une interface graphique. Il sera très intéressant lorsque vous aurez à créer des API et faire un plan de test (cours du semestre 4).

Pour vous entraîner vous pouvez suivre le lien suivant : <https://practicalprogramming.fr/debuter-avec-postman/>

Sites proposant ou référençant des API :

- <https://data.enedis.fr/explore/?sort=modified> (enedis)
- <https://public.opendatasoft.com/> (divers)
- <http://fr.openfoodfacts.org/data> (nourriture)
- <http://fr.openbeautyfacts.org/data> (produits cosmétiques)
- <http://api.open-notify.org/> (station spatiale internationale)
- <https://www.flickr.com/services/api/> (flicker)
- <https://www.imdb.com/> BDD de films
- <https://www.themoviedb.org/?language=fr> BDD de films
- Site qui référence des API Web : <https://www.programmableweb.com/>
- <https://github.com/public-apis/public-apis> (grosse liste)
- <https://api.gouv.fr/> API des services publics
- Twitter : <https://dev.twitter.com/rest/public>
- Facebook : <https://developers.facebook.com/>
- Instagram : <https://www.instagram.com/developer/>
- Spotify : <https://developer.spotify.com/web-api/>
- Insee: <https://api.insee.fr/catalogue/> et [pynsee](#)
- Pole Emploi : <https://www.emploi-store-dev.fr/portail-developpeur-cms/home.html>
- SNCF : <https://data.sncf.com/api>
- Banque Mondiale : <https://datahelpdesk.worldbank.org/knowledgebase/topics/125589>

4. Exemple d'utilisation de Postman

TRAVAIL A FAIRE
Tester l'API OpenWeatherMap sur la ville de **Poitiers** avec l'outil Postman. Une fois connecté à postman (création de compte obligatoire), vous n'avez juste qu'à rentrer l'URL avec la clé et le lieu (paramètres) :

GET ▼ <http://api.openweathermap.org/data/2.5/weather?APPID=369acb283342c0010fdeef5621dd1fc6&q=P...> Send ▼

Params ● Authorization Headers (5) Body Pre-request Script Tests Settings Cookies

Query Params

	KEY	VALUE	DESCRIPTION	...	Bulk Edit
☒	APPID	369acb283342c0010fdeef5621dd1fc6			
☒	q	Poitiers,fr			

Body Cookies Headers (9) Test Results 🌐 200 OK 151 ms 861 B Save Response ▼

Pretty Raw Preview Visualize JSON ▼ ↕

```

36     "sunrise": 1660798833,
37     "sunset": 1660849492
38   },
39   "timezone": 7200,
40   "id": 2986494,
41   "name": "Arrondissement de Poitiers",
42   "cod": 200

```